GROUP PROJECT

Advanced Analytics in action: Agent Skills and Multi-Agent Systems powered by Machine Learning

1. Overview and Learning Objectives

In this group project, you and your group will design, build, and critically evaluate a multi-agent AI system that combines machine learning with SKILL.md-governed agent behaviour.

Your work will be grounded in a real-world business context. You will be expected to connect the entire analytics pipeline, from raw data and model training, through evaluation and deployment, to the orchestration layer that ensures the system is practical and valuable in real use.

> **What makes this project distinctive**
>
> This is *not merely* a prompt-engineering or API-calling exercise. Each stage of the machine learning pipeline, from data preprocessing through model training, evaluation, selection, and inference, is governed by its own SKILL.md and executed as an agent node in a LangGraph pipeline. Downstream agent skills then consume the model output and deliver a business-facing result through a Gradio interface. You are expected to design, build, and explain the entire chain.

By the end of this project, you will be able to:

- Design a multi-agent pipeline in which each stage of a machine learning workflow is decomposed into a distinct agent skill governed by its own SKILL.md file.
- Train two or more candidate ML models on a domain-specific labelled dataset, evaluate them using appropriate metrics, and make a justified model selection decision.
- Author SKILL.md files for every pipeline stage, specifying the inputs received from upstream agent state, the processing performed, and the outputs passed downstream.
- Build downstream agent skills that consume model outputs and deliver actionable, business-facing results through a Gradio user interface.
- Evaluate model performance critically using metrics appropriate to the task type and interpret the results in the context of the business use case.
- Evaluate the full system end-to-end, including pipeline failure modes, model limitations, and real-world deployment considerations.
- Apply the module AI usage policy responsibly and document AI assistance with transparency.

2. The Machine Learning and Agent Skills requirement

This project treats the machine learning pipeline not as a single black box but as a sequence of inspectable stages. Each stage is implemented as a separate agent skill in the LangGraph pipeline and governed by its own SKILL.md file. This section defines the required pipeline structure, the ML component, and the downstream agent skills that deliver results to a business user.

2.1 The ML Pipeline as Agent Skills

Every group must decompose their machine learning workflow into individual agent skills, one per pipeline stage. Each skill must have its own SKILL.md file. The pipeline must progress logically from raw data through to a business-facing output, covering at minimum the following concerns: data preparation, model training across at least two candidate models or configurations, model evaluation and comparison, model selection

with a justified decision, inference on new inputs, and at least one downstream skill that translates the model output into something a non-technical business user can act on.

How groups structure and name their stages, what they pass through agent state, and how they connect the pipeline to the Gradio interface is a core design decision that groups are expected to make themselves. The SKILL.md files are where those decisions are documented and justified.

> **What agent state is for**
> Agent state is the shared memory that connects pipeline stages. Each skill reads what it needs from state and writes its outputs back to state for the next skill to consume. Designing a clean, well-typed state structure is part of the SKILL.md design challenge. Think carefully about what each stage needs and what it produces before writing any code.

2.2 The Machine Learning comparison requirement

The train-models and evaluate-models stages together must satisfy the following: at least two candidate models or configurations are trained on the same dataset, evaluated using the same metrics on the same held-out data, and compared fairly before a selection is made.

The purpose of comparing models is not to find the best number. It is to develop the habit of analytical reasoning about model choice: understanding why one model outperforms another on a given dataset, what the trade-offs are, and what the implications are for the business use case.

Note: *Simply calling a single pretrained model on a single input at inference time does not satisfy this requirement. There must be a labelled dataset, training loops for at least two candidate models or configurations, evaluation on held-out data, and a justified selection decision recorded in the select-model skill output.*

2.3 Downstream Agent Skills

After run-inference, the pipeline must include at least one downstream agent skill that takes the model prediction and produces something meaningful for a business user. This skill bridges the gap between a raw model output and a decision or recommendation that a non-technical user can act on.

The Gradio interface must display the output of the downstream skill, not raw model scores. A business user should be able to use the interface without knowing what model was used or what the prediction probabilities mean in technical terms.

2.4 Required evaluation metrics

The following table shows the minimum required metrics by task type. Groups must report all metrics listed for their task type in the technical report.

Task Type	Required Metrics	Recommended Visualisation

Binary Classification	Accuracy, Precision, Recall, F1-score, AUC-ROC	Confusion matrix, ROC curve
Multi-class Classification	Per-class Precision, Recall, F1-score, Macro F1, Weighted F1	Confusion matrix heatmap, per-class bar chart
Regression	RMSE, MAE, R-squared	Predicted vs actual scatter plot, residual plot
Clustering / Unsupervised	Silhouette score, Davies-Bouldin index, Inertia (if k-means)	Elbow plot, cluster scatter plot
Ranking / Retrieval	Precision@k, Recall@k	Precision-recall curve

3. Example use cases and suggested ML approaches

The examples below are provided to inspire ideas, not to restrict them. Groups are free to propose any business use case, including domains not listed here, as long as it satisfies the project requirements. There is no fixed list to select from.

Each row shows an example business domain, a suggested model comparison, a dataset source, and an example of how the full agent skill pipeline might be structured from preprocessing through to the business-facing downstream output. The pipeline column shows the skill chain in execution order, with the inference stage marked with an asterisk.

#	Business Domain	Suggested ML Comparison	Suggested Dataset	Example Agent Skill Pipeline (* = inference stage)
1	Customer Support Triage	Compare logistic regression, random forest, and gradient boosting for multi-class complaint classification using TF-IDF features	Kaggle Customer Support on Twitter, or a synthetic labelled complaint dataset	preprocess-data → train-models → evaluate-models → select-model → *classify-complaint → lookup-order-record → draft-response
2	Financial Anomaly Detection	Compare Isolation Forest and One-Class SVM for anomaly detection on transaction features	UCI Credit Card Fraud dataset or synthetic invoices	preprocess-data → train-models → evaluate-models → select-model → *detect-anomaly → extract-document-context → flag-compliance-risk
3	Recruitment Screening	Compare logistic regression and gradient boosting for binary CV-to-role fit scoring on	Kaggle HR Analytics dataset or synthetic CV features	preprocess-data → train-models → evaluate-models → select-model → *score-candidate-fit → parse-cv-text → draft-outreach-email

		structured features		
4	Retail Out-of-Stock Detection	Compare two pretrained CNN feature extractors as frozen providers with a trained classification head	SKU110K dataset or Roboflow retail shelf datasets	preprocess-data → train-models → evaluate-models → select-model → *detect-shelf-gap → check-planogram-compliance → generate-restock-order
5	Legal Clause Classification	Compare TF-IDF with SVM against TF-IDF with random forest for clause type prediction	CUAD (Contract Understanding Atticus Dataset)	preprocess-data → train-models → evaluate-models → select-model → *classify-clause → compare-to-template → summarise-risk
6	Medical Image Classification	Compare two pretrained CNNs as frozen feature extractors with a trained classification head	NIH Chest X-Ray or RSNA Pneumonia Detection datasets	preprocess-data → train-models → evaluate-models → select-model → *classify-scan-finding → cross-reference-patient-history → draft-radiologist-note
7	Content Policy Violation	Compare TF-IDF with gradient boosting against word embeddings with logistic regression for toxicity classification	Jigsaw Toxic Comment dataset (Kaggle)	preprocess-data → train-models → evaluate-models → select-model → *classify-violation → assess-severity → recommend-moderation-action
8	Supply Signal Classification	Compare TF-IDF with logistic regression against TF-IDF with random forest for supply chain signal categorisation	Custom labelled news headlines or GDELT dataset	preprocess-data → train-models → evaluate-models → select-model → *classify-supply-signal → extract-affected-entities → draft-procurement-alert
9	Property Price Prediction	Compare linear regression, random forest regression, and gradient boosting on structured property features	Kaggle House Prices or HDB resale price dataset	preprocess-data → train-models → evaluate-models → select-model → *predict-price → benchmark-against-comparables → write-listing-copy
10	HR Query Intent Detection	Compare a bag-of-words classifier against a TF-IDF classifier for intent	Custom labelled HR FAQ dataset (groups may	preprocess-data → train-models → evaluate-models → select-model → *detect-intent → answer-policy-

		routing on HR FAQ data	generate synthetic data)	question → generate-equipment-request
11	Insurance Fraud Detection	Compare XGBoost and LightGBM on structured insurance claim features	Kaggle Insurance Fraud Detection dataset	preprocess-data → train-models → evaluate-models → select-model → *score-fraud-risk → verify-policy-coverage → draft-settlement-letter
12	Crop Disease Classification	Compare two pretrained CNN feature extractors with a trained classification head on labelled crop images	PlantVillage dataset (Kaggle)	preprocess-data → train-models → evaluate-models → select-model → *classify-crop-disease → estimate-affected-area → recommend-treatment-plan
13	Cybersecurity Threat Triage	Compare random forest and gradient boosting for multi-class threat categorisation from structured alert features	CICIDS or UNSW-NB15 network intrusion dataset	preprocess-data → train-models → evaluate-models → select-model → *classify-threat → assess-blast-radius → generate-response-playbook
14	Competitive Signal Tracking	Compare TF-IDF with logistic regression against TF-IDF with random forest for strategic signal categorisation	Custom labelled news dataset or RCV1 corpus	preprocess-data → train-models → evaluate-models → select-model → *classify-signal → score-competitive-threat → generate-briefing-note
15	Financial Trend Prediction	Compare linear regression against a gradient boosting regressor on historical financial KPI features	Company financial filings via SEC EDGAR or Yahoo Finance API	preprocess-data → train-models → evaluate-models → select-model → *predict-kpi-trend → summarise-financial-performance → draft-management-commentary

4. Technical requirements

4.1 Minimum system specification

Every submission must include all of the following:

1. A LangGraph pipeline in which each stage of the ML workflow is a distinct agent node: preprocess-data, train-models, evaluate-models, select-model, and run-inference at minimum.
2. At least one downstream agent skill beyond run-inference that consumes the model prediction and produces a business-facing output for display in the Gradio interface.

3. One SKILL.md file per pipeline stage. Every SKILL.md must include YAML frontmatter (name, description), a When to use section, a How to execute section, an Inputs from agent state section, an Outputs to agent state section, an Output format section, and relevant notes.
4. At least two candidate models or configurations trained and compared within the train-models and evaluate-models stages on a labelled dataset, with a justified selection recorded in the select-model stage output.
5. A model evaluation section in the notebook that reports all required metrics for the task type as listed in Section 2.4, with at least one visualisation.
6. A Gradio user interface that exposes the downstream skill output to a non-technical business user. The interface must not require the user to understand model internals.
7. A working demonstration delivered as a Jupyter notebook (.ipynb) runnable in Google Colab.

4.2 SKILL.md quality standard

SKILL.md files are the policy layer of the system.

- Every SKILL.md must specify what the skill does, what inputs it receives from upstream agent state, what outputs it passes downstream, and how a developer would execute it.
- The SKILL.md for the train-models stage must list the candidate models and configurations.
- The SKILL.md for evaluate-models must state the metrics computed and the visualisations produced.
- The SKILL.md for select-model must describe the decision logic used to justify the selection.
- Downstream skill SKILL.md files must describe how the model prediction is translated into the business-facing output.

4.3 Recommended stack

- Python 3.10 or above
- PyTorch or scikit-learn for model training and evaluation
- pandas and numpy for data loading and preprocessing
- matplotlib or seaborn for evaluation visualisations
- LangGraph for the agent pipeline
- OpenAI GPT-4o-mini or equivalent model for agent skills
- Gradio for the user interface
- Google Colab (GPU runtime recommended if using PyTorch for image models)
- Google Secrets for API key management

5. Deliverables

Each group must submit the following four deliverables as **a single ZIP archive** by the deadline.

Deliverable 1: Working Jupyter Notebook

A single .ipynb file runnable end-to-end in Google Colab. The notebook must be clearly structured with labelled cells covering each pipeline stage: data loading and preprocessing, training of at least two candidate models, evaluation with metrics and visualisations, model selection with written justification, inference on a

new input, downstream skill output, LangGraph pipeline definition with all SKILL.md content, and Gradio interface launch with share=True. No hardcoded API keys.

Deliverable 2: SKILL.md files

All SKILL.md files submitted as individual .md files in a folder named skills/ within the ZIP archive, one file per pipeline stage. Each file must match the quality standard in Section 4.2. Files must be identical to those used in the notebook.

Deliverable 3: Technical report

A written report of 2500 to 3000 words (excluding figures, tables, code snippets, and references) covering:

- System Architecture: LangGraph pipeline diagram, explanation of each agent node, and how SKILL.md files govern routing and execution.
- Business Case: Target business context, problem being solved, expected value delivered, and identified stakeholders.
- Data and ML Methodology: Dataset description, preprocessing steps, the candidate models or configurations compared, training setup, evaluation methodology, and justification of the final model selection.
- Model Evaluation: Full results table with all required metrics, visualisations, and a critical interpretation of what the numbers mean for the business use case.
- SKILL.md Design Rationale: Design decisions for each pipeline stage SKILL.md, explaining how the inputs and outputs were chosen, how the stage connects to adjacent stages, and how the downstream skill translates model output into business language.
- System Evaluation: At least three end-to-end test cases with documented inputs, expected outputs, and actual outputs. Discussion of failure modes and limitations.
- AI Usage Declaration: Transparent account of AI tool use per Section 7.

Deliverable 4: Slides and group presentation

A 20-minute pre-recorded video presentation. All group members must appear on camera and contribute to the recording. The presentation must include a screen recording of the Gradio interface, a walkthrough of the model comparison results and evaluation metrics, and an explanation of how each pipeline stage connects as an agent skill. Submit the presentation slides in PowerPoint format (.pptx) as part of the ZIP archive. **Do not submit the video file itself. Include the video hyperlink in the technical report.**

6. Assessment Rubrics

The project is marked out of 100 points across six components. The rubric below describes the criteria for each grade band.

Component 1: Machine Learning Development and Evaluation (30 points)

This is the highest-weighted component and reflects the centrality of ML to the BT5151 module.

Criterion	Distinction (85-100%)	Merit (70-84%)	Pass (50-69%)	Fail (0-49%)

Dataset Selection and Preparation	Dataset is well-chosen for the business context. Preprocessing is thorough and justified, including handling of missing values, class imbalance, and train/validation/test splits.	Dataset is appropriate with adequate preprocessing. Minor gaps in justification.	Dataset is used but preprocessing is minimal or poorly explained.	Dataset is unsuitable, missing, or not pre-processed.
Model Comparison and Training	At least two candidate models or configurations trained and evaluated fairly. Comparison is rigorous, reproducible, and well-documented. Model selection is clearly justified by the results.	Two models compared with adequate documentation. Selection justified but reasoning lacks depth.	Two models present but comparison is superficial or selection is not justified by the evidence.	Only one model trained, or no genuine comparison performed.
Metric Reporting	All required metrics for the task type reported correctly on held-out data. Results are clearly presented and correctly interpreted.	Most required metrics reported with minor errors or gaps in interpretation.	Some metrics reported but key ones are missing or computed incorrectly.	Metrics absent or only reported on training data.
Visualisation Quality	At least one high-quality, properly labelled visualisation (confusion matrix, ROC curve, loss curve, etc.) that supports the evaluation narrative.	Visualisation present but labelling or interpretation is incomplete.	Basic plot included with minimal explanation.	No visualisation included.
Critical Interpretation	Metrics are interpreted in business terms. Limitations of the model are acknowledged honestly. Results are compared to a baseline.	Some business interpretation present. Baseline comparison attempted.	Results reported without meaningful interpretation.	No critical analysis of results.

Component 2: SKILL.md Design Quality (20 points)

Criterion	Distinction (85-100%)	Merit (70-84%)	Pass (50-69%)	Fail (0-49%)
Completeness	All required sections present for every pipeline stage. Every SKILL.md includes inputs from agent state, outputs to agent state, and execution instructions.	All sections present with minor omissions across one or two skills.	Most sections present but inputs/outputs or execution steps sparse or missing.	SKILL.md files are incomplete, missing key sections, or absent for some stages.
Pipeline State Specification	Every SKILL.md precisely specifies what it reads from agent state and what it writes back. The chain of inputs and outputs is coherent and unambiguous across all stages.	Most stages specify inputs and outputs clearly. Minor gaps or ambiguities in one or two stages.	Some stages specify inputs and outputs, but the state chain is incomplete or inconsistent.	Inputs and outputs not specified, or agent state is not used to connect stages.
Business Readability	A non-technical domain professional could read any SKILL.md and understand what that stage produces	Mostly clear with occasional technical jargon that limits readability.	Readable by a technical audience but not by business users.	Unclear or written only as code comments.

	and why it matters for the business outcome.			
Downstream Skill Quality	Downstream SKILL.md clearly describes how the model prediction is translated into a business-facing output. The output format is specific and matches what is shown in the Gradio interface.	Downstream skill described but the translation from prediction to business output is thin.	Downstream skill present but output format is vague or does not reflect actual interface output.	No downstream skill SKILL.md, or downstream skill is indistinguishable from the inference stage.

Component 3: Technical Implementation (20 points)

Criterion	Distinction (85-100%)	Merit (70-84%)	Pass (50-69%)	Fail (0-49%)
Pipeline Correctness	LangGraph pipeline runs end-to-end without errors across all tested inputs. ML model output is correctly passed through agent state.	Pipeline runs for most inputs with minor errors on edge cases.	Pipeline runs for the demo case but breaks on varied inputs.	Pipeline does not run or produces incorrect outputs.
ML and Pipeline Integration	ML model output is meaningfully used by downstream agents. The synthesiser combines model confidence, predictions, and other tool results coherently.	Integration works but ML output is used superficially by downstream agents.	ML model runs but its output is not properly consumed by the pipeline.	ML model is disconnected from the agent pipeline.
Pipeline Stage Coverage	All required pipeline stages present and functional. Each stage is a distinct agent node with its own SKILL.md. Agent state correctly carries data between stages.	All stages present but one or two share state awkwardly or are not fully distinct.	Some stages merged or missing. Agent state not consistently used to connect stages.	Pipeline is not decomposed into stages, or fewer than the minimum stages are implemented.
Gradio Interface and Code Quality	Interface is intuitive, shows ML predictions and confidence alongside final answer, and works reliably via share=True. Code is clean and well-commented.	Interface works with minor UX issues. Code is readable.	Interface launches but output display is unclear. Code is difficult to follow.	Interface non-functional or code is disorganised.

Component 4: Business Case Analysis (15 points)

Criterion	Distinction (85-100%)	Merit (70-84%)	Pass (50-69%)	Fail (0-49%)
Business Problem and Value	Problem is specific and well-researched. Value proposition is substantiated with reference to what changes in the business when the system is deployed.	Problem and value clearly stated but lacking depth.	Problem stated but remains generic.	No credible business context.

ML Relevance to Business Problem	Clear explanation of why ML is the right tool for this problem and how model performance directly affects business outcomes.	ML relevance described but connection to business outcomes is thin.	ML included but its business relevance is not explained.	No justification for using ML in this context.

Component 5: Critical Reflection and Evaluation (10 points)

Criterion	Distinction (85-100%)	Merit (70-84%)	Pass (50-69%)	Fail (0-49%)
End-to-End System Tests	Three or more complete runs of the full pipeline documented, including at least one challenging or unexpected input (for example, an ambiguous query, an edge case, or an input the model handles poorly). For each run: the input is shown, the expected business output is stated, the actual output is recorded, and any model confidence scores are noted.	Three complete runs documented but all use straightforward, ideal inputs where the system is expected to succeed.	Fewer than three runs, or runs are present but inputs, outputs, or confidence scores are not fully documented.	No documented pipeline runs, or outputs are copy-pasted without any commentary.
What went wrong and why	At least two things the system gets wrong are identified from actual pipeline runs, not hypothetically. For each: what the failure looks like (for example, the model consistently misclassifies a particular category, or a downstream skill produces a vague output when confidence is low), why it happens, and what the consequence would be for a real business user of the system.	Failures identified from actual runs with some explanation of why they occur, but the consequence for the business user is not addressed.	Failures are mentioned but described in vague terms such as "the model sometimes gets it wrong" without identifying patterns or causes.	No failures discussed, or only theoretical limitations copied from a model documentation page without any connection to the group's own system.
Reflection and Improvement	Thoughtful reflection on what was learned about both ML and agent system design. Specific, feasible improvements proposed for both the model and the pipeline.	Reflection present with some improvement ideas.	Generic reflection with little specificity.	No reflection.

Component 6: Presentation and Communication (5 points)

Criterion	Distinction (85-100%)	Merit (70-84%)	Pass (50-69%)	Fail (0-49%)

Recorded Demonstration	Demonstration shows the Gradio UI, ML model outputs, and pipeline routing for at least one task.	Demonstration works with minor issues.	Demonstration is partial or poorly planned.	Demonstration fails.
Clarity, Participation, and Duration	All group members appear on camera and contribute meaningfully to the recording. ML methodology and pipeline design are explained clearly enough for a non-specialist to follow. Recording is within 20 minutes.	Most members appear and contribute. Explanation is mostly clear. Recording is within 2 minutes of the limit.	Participation is uneven with one or two members absent from the recording, or the recording exceeds the limit by more than 1 minute.	Only one member appears in the recording, or the recording is significantly under or over the time limit.

Grade Summary

Component	Points available
1. Machine Learning Development and Evaluation	30
2. SKILL.md Design Quality	20
3. Technical Implementation	20
4. Business Case and Ethical Analysis	15
5. Critical Reflection and Evaluation	10
6. Presentation and Communication	5
Total	**100**

Important weighting note

Component 1 (Machine Learning) carries the highest single weighting at 30 points. A submission that trains only one model without comparison, or that uses a pretrained model at inference time only without any training loop or evaluation on labelled data, cannot achieve more than 70 points in total regardless of the quality of the other components.

7. AI Usage Policy

7.1 Policy principles

This project takes place in the context of a coursework module on advanced analytics and machine learning. AI tools are part of the professional environment students are preparing to enter. The policy is designed to develop responsible and transparent AI use rather than prohibit it.

The core principle is this: AI tools may support your work, but the intellectual contribution, design decisions, model choices, evaluation interpretation, and business judgement must be your own. A group that relies on AI to generate its work without genuine understanding will demonstrate that clearly in the recorded presentation and will score poorly on every rubric component.

7.2 What is permitted

- Using AI coding assistants (such as Cursor, Claude, or ChatGPT) to help write, debug, or refactor code, provided the group understands every part of the code and can explain it.
- Using AI to improve the clarity or grammar of text the group has already written in full.
- Using AI to check that a completed SKILL.md is readable and unambiguous. The content, structure, and design decisions must come entirely from the group.
- Using AI to generate example inputs for testing the pipeline or to suggest edge cases to consider.
- Using AI to research background information on a business domain, dataset, or algorithm, provided the group independently verifies any facts or claims before including them in the report.

7.3 What is NOT permitted

- Using AI to select, download, or preprocess a dataset without the group understanding what the data contains, how it is structured, and why it is appropriate for the business problem.
- Using AI to decide which models to train, how to configure them, or how to interpret the evaluation results. These are the core analytical decisions the project is assessing.
- Using AI to generate any part of the technical report, SKILL.md files, or presentation script. All written and spoken content must originate from the group.
- Using AI to generate or narrate any part of the recorded presentation.
- Misrepresenting the extent of AI use in the AI Usage Declaration.

7.4 AI usage declaration requirement

Every group must include an AI Usage Declaration as a dedicated section of the technical report describing:

1. Which AI tools were used (name and version where known).
2. For what purpose each tool was used, including any use in model selection, training configuration, or evaluation interpretation.
3. How the group critically engaged with or revised the AI output before including it in the submission.
4. An honest assessment of how much the final submission reflects AI-generated versus group-generated content, particularly for the ML component.

Declarations will not affect the mark directly. Honesty is expected and valued. Misrepresentation of AI use is treated as an academic integrity violation under the standard module policy.

7.5 A practical note

The strongest submissions will use AI to handle boilerplate and syntax, freeing the group to invest more time in the choices that matter: which model architecture is right for this problem, what the confusion matrix is telling you about model behaviour, how model confidence should influence agent routing, and what the failure modes mean for the real business users of the system. That analytical judgement is what BT5151 is assessing. It cannot be delegated to an AI.

8. Submission Checklist

Before submitting, confirm that your ZIP archive contains all of the following:

#	Item	Included
1	Jupyter notebook (.ipynb) runnable in Google Colab with GPU runtime	[ ]
2	One SKILL.md file per pipeline stage in a skills/ folder	[ ]
3	Technical report (2500 to 3000 words) as a PDF document	[ ]
4	Presentation slides (pptx format)	[ ]
5	No hardcoded API keys anywhere in the submission (Use Google Secrets instead)	[ ]
6	Model evaluation section with all required metrics and at least one visualisation	[ ]
7	At least two candidate models compared with evaluation metrics on held-out data and model selection justified in the select-model stage	[ ]
8	AI Usage Declaration included in the technical report	[ ]
9	All group member names and student IDs on the report cover page	[ ]

9. Timeline

Week	Milestone	Details
Week 7	Idea Proposal and Planning	Develop your own business use case idea or draw inspiration from the examples in Section 3. Begin background research on the business context and identify a suitable dataset.
	Data Exploration and Preprocessing	Download and explore the dataset. Perform exploratory data analysis. Preprocess data and prepare train/validation/test splits. Document all pre-processing decisions.
Week 8 to 9	Model Training or Evaluation	Train at least two candidate models or configurations on the prepared dataset. Evaluate each on held-out data and record all required metrics. Begin drafting the model selection justification and the ML-backed SKILL.md.
	Model Evaluation and Pipeline Design	Complete full model evaluation with all required metrics and visualisations. Write the model selection justification. Draft all pipeline SKILL.md files specifying inputs, outputs, and execution for each stage. Design the LangGraph graph structure on paper before writing agent code.
Week 10 to 12	Agent Pipeline Build	Implement all LangGraph pipeline stages as agent nodes following the SKILL.md specifications. Integrate the trained model into the run-inference stage. Build at least one downstream skill. Get the notebook running end-to-end in Colab with a basic Gradio interface showing downstream output.

	Testing and Refinement	Run end-to-end test cases including adversarial cases. Refine SKILL.md files based on observed behaviour. Improve Gradio UI to surface ML confidence scores.
Week 13	Report and Slides	Write the technical report with all required sections. Prepare presentation slides. Record the group video presentation including a screen recording of the Gradio demo and a walkthrough of the model evaluation results.
	Submission and Presentation	Submit the ZIP archive by 11:59 PM on the submission deadline. Include the hyperlink to your pre-recorded presentation video inside the technical report. Do not submit the video file itself.

10. Academic Integrity

10.1 Academic integrity

All work submitted must be the group's own. Code, SKILL.md files, trained model weights, and written content copied from other groups or from public repositories without attribution will be treated as academic misconduct. Similarities between submissions will be checked. Submitting a model trained by someone else as your own work is a serious violation.

Each group must work independently. Discussing general concepts from the module with peers is acceptable, but any collaboration beyond this is NOT permitted. Sharing code, model weights, SKILL.md files, or written sections between groups is NOT permitted.

10.3 Late submission

Late submissions will be penalised at 10 percentage points for each 24-hour period past the deadline, up to a maximum of 48 hours, unless an extension has been agreed in writing with the lecturer before the deadline.

Extensions will only be granted in exceptional circumstances with supporting documentation. Personal workload, technical difficulties, and competing deadlines do NOT qualify.

Time Submitted	Penalty	Final Grade Calculation
Up to 24 hours late	10%	(Score) × 0.90
24 to 48 hours late	20%	(Score) × 0.80
Over 48 hours late	100%	No submissions accepted (Zero grade)

11. Peer Evaluation

Upon submission, every group member must complete an individual peer evaluation using the TEAMMATES platform. You will be asked to rate each of your groupmates on their contribution to the project and provide brief written comments.

Your ratings and comments will never be shared with your groupmates. TEAMMATES aggregates responses anonymously and only the instructor will have access to individual submissions. Be honest. Peer evaluation exists to ensure that the final grade reflects each member's actual contribution, not just the group's collective output.

Stop thinking like a student. Start thinking like a builder.